

Tutorial - Semana 5, Encontro 1

Introdução

A Semana 4 encerrou com dois Autoloads funcionais — `GameManager` e `SaveManager` — sustentando estado de partida e persistência entre cenas. Nada desse trabalho é alterado agora. Este encontro resolve um problema diferente: como um objeto do mundo (uma porta, uma alavanca) responde a uma chamada do Player, sem que os dois precisem se conhecer diretamente. A resposta é o contrato `Interactable`, que passa a sustentar todo objeto interativo do Vertical Slice até a Semana 17, incluindo a ampliação prevista para a Semana 10 (conexão da interação ao Inventário).

Objetivos da semana

- Explicar o contrato `Interactable` como mecanismo de comunicação desacoplada entre Player e objetos do mundo.
- Diferenciar comunicação por contrato de comunicação por referência direta (`get_node`).
- Criar um contrato `Interactable` genérico e implementá-lo em uma Scene do Vertical Slice.

Resultado esperado ao final da semana

Ao final da Semana 5 (Encontros 1 e 2), cada estudante terá, sobre o `GameManager` e `SaveManager` herdados da Semana 4, um contrato `Interactable` implementado por ao menos uma Scene do Vertical Slice, com um Signal disparado por interação e conectado a uma reação concreta. Este tutorial cobre apenas o **Encontro 1**: a criação do contrato `Interactable`, sua implementação em uma Scene (`Door.tscn`) e a detecção de proximidade via `InteractionComponent` no Player.

Pré-requisitos

- Projeto do Vertical Slice com `GameManager` e `SaveManager` registrados como Autoload (ver Tutorial - Semana 4, Encontro 2).
 - Input Map configurado desde a Semana 2, com liberdade para adicionar uma nova ação (`interagir`).
-

Antes de começar

O que o estudante deverá possuir antes desta semana

- O projeto do Vertical Slice completo até a Semana 4, com `GameManager` e `SaveManager` registrados e testados.

Arquivos necessários

- Nenhum arquivo externo. O contrato `Interactable`, a Scene `Door.tscn` e o `InteractionComponent` são criados dentro do próprio projeto.

Assets utilizados

- Um mesh simples para representar a porta (pode ser um `MeshInstance3D` com `BoxMesh`, ou um asset do Kenney Dungeon Kit já disponível no projeto, conforme `PROJECT_ARCHITECTURE.md`, seção 8).

Projeto esperado

- Projeto aberto no Godot 4.7, com `level_exploration.tscn` funcionando e os dois Autoloads da Semana 4 ativos.
- Pastas `scenes/interactables/` e `scripts/components/` criadas (ou prontas para ser criadas), conforme a estrutura do `PROJECT_ARCHITECTURE.md` (seção 8).

Imagem sugerida

Objetivo: mostrar a estrutura de pastas do projeto com `scenes/interactables/` e `scripts/components/` em destaque. Enquadramento: captura de tela do FileSystem Dock do Godot, árvore de pastas expandida. Elementos importantes: pastas `scenes/interactables/`, `scripts/components/`, `scripts/autoload/` (já existente). Destaque: as duas pastas novas desta semana. Legenda sugerida: "Novas pastas do projeto para interativos e Components, Semana 5."

Parte 1 — O problema da comunicação acoplada

Objetivo

Entender por que o Player não deve conhecer o tipo concreto de cada objeto interativo, antes de escrever qualquer contrato.

Conceito

Toda engine de jogos com múltiplos sistemas independentes precisa resolver o mesmo problema: como um sistema chama outro sem depender do tipo concreto dele. Se o Player precisasse conhecer explicitamente a classe `Door`, `Lever`, `Chest` e cada novo objeto interativo criado depois, cada adição exigiria alterar o código do Player — um acoplamento que não escala e que se tornaria insustentável já na Semana 10, quando a interação passar a se conectar ao Inventário.

O contrato `Interactable` resolve isso definindo apenas um comportamento esperado — responder a `interact()` — sem exigir uma hierarquia de herança comum entre os objetos que o implementam. O Godot expressa esse contrato via **duck typing**, checando `has_method("interact")` em tempo de execução, ou, no Orchestrator, por um nó de interface dedicado, sem precisar de uma linguagem de interfaces formal como a Unity/C# usa.

Passo a passo

Esta parte não possui etapas de implementação no editor — é a base conceitual discutida antes da Parte 2.

1. Revisar com a turma como `GameManager` e `SaveManager` já resolvem estado global e persistência (Semana 4).
2. Perguntar: "se o Player precisar reconhecer uma porta, uma alavanca e um baú, o que muda no código do Player a cada novo tipo?"
3. Concluir que a resposta correta é "nada deveria mudar" — e que essa é exatamente a garantia que um contrato oferece.

Resultado esperado

A turma reconhece o acoplamento direto (Player conhecendo cada classe concreta) como um problema de escala, e entende que um contrato resolve isso sem herança compartilhada.

Verificando

1. Confirmar que os estudantes conseguem explicar, com suas próprias palavras, por que `get_node` direto para cada tipo de objeto interativo não escala.

Problemas comuns

- Confundir "contrato" com "classe base"/herança: reforçar que `Interactable` não é uma classe que `Door` estende, é apenas um método esperado que qualquer Node pode implementar.

Boas práticas

- Manter essa discussão breve, reservando a maior parte do encontro para a implementação nas Partes 2 e 3.

Comparação com Unity

A Unity resolve o mesmo problema com `interface` formal de C# (`IInteractable`), implementada por qualquer `MonoBehaviour` que declare os métodos exigidos — uma checagem de tipo em tempo de compilação. O Godot, ao usar duck typing via `has_method` , faz a mesma checagem em tempo de execução: mais flexível, mas sem a garantia do compilador de que o método existe antes de rodar.

Parte 2 — Criando o contrato `Interactable` e a `Scene Door`

Objetivo

Implementar o contrato `Interactable` em uma Scene concreta do Vertical Slice.

Conceito

Um contrato via duck typing no GDScript não exige um arquivo de "interface" separado: qualquer script que declare uma função `interact()` já satisfaz o contrato, e qualquer outro script pode checar essa capacidade com `has_method("interact")` antes de chamar a função, sem precisar saber o tipo concreto do Node. A primeira Scene a implementar esse contrato é `Door.tscn` — um objeto simples, cuja reação inicial (nesta parte) é apenas confirmar que a chamada chegou.

Passo a passo

1. No FileSystem Dock, crie a pasta `scenes/interactables/`, caso ainda não exista, conforme o `PROJECT_ARCHITECTURE.md` (seção 8).
2. Dentro de `scenes/interactables/`, crie uma nova Scene com nó raiz `StaticBody3D`, renomeado para `Door`.
3. Adicione como filho um `MeshInstance3D` com um `BoxMesh`, representando visualmente a porta, e um `CollisionShape3D` correspondente.
4. Salve a Scene como `Door.tscn` dentro de `scenes/interactables/`.
5. Com o nó raiz `Door` selecionado, clique em **Attach Script**, mantenha o caminho sugerido (`door.gd`) e clique em **Create**.
6. No topo do script, adicione `class_name Door`.
7. Declare a função do contrato:

```
func interact() -> void:
    print("Door: interagido")
```

8. Salve o script (**Ctrl+S**) e a Scene (**Ctrl+S**).

Resultado esperado

Existe uma Scene `scenes/interactables/Door.tscn`, com script `door.gd` implementando `interact()`, que imprime uma mensagem de confirmação quando chamada.

Verificando

1. Abra o script `door.gd` e confirme que a função `interact()` está declarada corretamente, sem erros de sintaxe.
2. Instancie `Door.tscn` dentro de `level_exploration.tscn` temporariamente e, no `_ready()` da própria Door (apenas para teste rápido), chame `interact()` uma vez — confirme a mensagem no Output.

3. Remova a chamada de teste do `_ready()` antes de seguir para a Parte 3, mantendo apenas a função `interact()`.

Problemas comuns

- Nomear a função de forma diferente de `interact` (por exemplo, `interagir` ou `on_interact`): o contrato desta disciplina usa `interact()` como nome fixo — qualquer outro nome quebra a checagem via `has_method("interact")` feita pelo Player.
- Implementar a lógica de reação fora da função `interact()`, por exemplo direto no `_process()`: a reação deve acontecer dentro de `interact()`, chamada explicitamente por quem detecta a interação.

Boas práticas

- Manter `Door.tscn` dentro de `scenes/interactables/`, seguindo a estrutura do `PROJECT_ARCHITECTURE.md` — a mesma pasta receberá `Lever.tscn`, `Chest.tscn` e `Checkpoint.tscn` nas próximas semanas.
- Nomear a Scene pela função que ela representa (`Door.tscn`), nunca por sua implementação (`InteractableBox01.tscn`), conforme a convenção de nomenclatura do projeto (seção 9 do `PROJECT_ARCHITECTURE.md`).

Comparação com Unity

Na Unity, o mesmo resultado exigiria declarar uma `interface IInteractable` com a assinatura do método `Interact()`, e a classe `Door` (um `MonoBehaviour`) implementaria essa interface explicitamente na declaração da classe (`public class Door : MonoBehaviour, IInteractable`). O Godot dispensa essa declaração explícita: basta que `door.gd` declare uma função chamada `interact`.

Parte 3 — Detectando o Interactable a partir do Player

Objetivo

Construir um `InteractionComponent` simples no Player, capaz de detectar um `Interactable` próximo e chamar `interact()` sem conhecer o tipo concreto do objeto.

Conceito

O contrato `Interactable` só é útil se algo o chamar. Esse "algo" é o `InteractionComponent`, um Component dedicado do Player (conforme `PROJECT_ARCHITECTURE.md`, seção 7) que usa uma `Area3D` para detectar objetos próximos e, ao receber a ação de interagir, checa `has_method("interact")` no objeto detectado antes de chamá-lo. Esse é o ponto exato em que o desacoplamento discutido na Parte 1 se torna código: o `InteractionComponent` nunca escreve `if objeto is Door`, apenas confirma que o objeto responde ao contrato.

Passo a passo

1. Abra **Project > Project Settings > Input Map** e adicione uma nova ação chamada `interagir`, associada a uma tecla livre (por exemplo, **E**).
2. No FileSystem Dock, crie a pasta `scripts/components/`, caso ainda não exista.
3. Na Scene do Player, adicione um novo nó filho `Area3D`, renomeado para `InteractionComponent`, com um `CollisionShape3D` filho definindo um raio pequeno de detecção.
4. Com `InteractionComponent` selecionado, anexe um script em `scripts/components/interaction_component.gd`, com `class_name InteractionComponent` no topo.
5. Declare uma variável para guardar o Interactable mais próximo detectado, por exemplo `var interactable_atual: Node = null`.
6. Conecte o sinal `body_entered` da própria `Area3D` a uma função `_on_body_entered(body: Node3D)`, que verifica `if body.has_method("interact"): interactable_atual = body`. Use sempre `body_entered` (nunca `area_entered`): a `Door` foi criada na Parte 2 com nó raiz `StaticBody3D`, e `area_entered` só dispara quando outro `Area3D` entra na zona — um `StaticBody3D` é sempre detectado via `body_entered`.

7. Conecte também o sinal `body_exited` a uma função `_on_body_exited(body: Node3D)` que limpa `interactable_atual = null` quando o objeto sai da área.
8. No script do Player (`player.gd`), no `_input()` ou `_unhandled_input()` , adicione a checagem da ação `interagir` : se pressionada e `InteractionComponent.interactable_atual` não for nulo, chame `interactable_atual.interact()` .
9. Rode `level_exploration.tscn` (F6), aproxime o Player da `Door.tscn` instanciada no nível e pressione **E**.
10. Confirme no Output a mensagem `"Door: interagido"` .

Resultado esperado

O Player detecta a `Door` ao se aproximar (via `Area3D`) e, ao pressionar a ação `interagir` , chama `interact()` corretamente — sem que o script do Player ou do `InteractionComponent` mencione a palavra `Door` em nenhum momento.

Verificando

1. Aproxime o Player de `Door.tscn` e confirme que `interactable_atual` deixa de ser nulo (pode ser testado com um `print` temporário).
2. Pressione **E** dentro da área de detecção e confirme a mensagem no Output.
3. Afaste o Player e confirme que `interactable_atual` volta a ser nulo, e que pressionar **E** fora da área não gera nenhuma chamada.
4. Instancie uma segunda `Door.tscn` em outro ponto do nível e repita o teste, confirmando que o `InteractionComponent` funciona com qualquer instância, não apenas a primeira.

Problemas comuns

- Chamar `interact()` sem checar `has_method("interact")` antes: se qualquer outro `body` sem esse método entrar na zona de detecção, a chamada direta geraria erro — sempre checar antes de chamar.
- Usar `area_entered` em vez de `body_entered` : a `Door` é um `StaticBody3D` , não um `Area3D` — `area_entered` nunca dispara para ela, e o `InteractionComponent` nunca detectaria a porta.
- Esquecer de limpar `interactable_atual` no `body_exited` : o Player continuaria chamando `interact()` em um objeto do qual já se afastou.
- Anexar o script de detecção ao nó errado (ao invés da `Area3D` do `InteractionComponent` , ao `CollisionShape3D` filho): sinais de área devem ser conectados a partir do nó `Area3D` , não do seu `CollisionShape3D` .

Boas práticas

- Manter o `InteractionComponent` limitado a detectar e chamar o contrato — nenhuma lógica de reação específica de `Door`, `Lever` ou `Chest` deve viver aqui.
- Testar sempre com pelo menos duas instâncias diferentes do objeto interativo antes de considerar a etapa concluída.

Comparação com Unity

Na Unity, esse mesmo papel seria cumprido por um script equivalente anexado ao Player, usando `OnTriggerEnter / OnTriggerExit` com um `Collider` marcado como `Trigger`, checando `if (other.TryGetComponent<IInteractable>(out var interactable))` antes de chamar `interactable.Interact()`. O padrão é o mesmo — detectar por proximidade e checar o contrato antes de chamar —, mudando apenas a forma de checagem:

`TryGetComponent<IInteractable>` (tipagem estática) na Unity, `has_method("interact")` (tipagem dinâmica) no Godot.

Ao final do encontro

Ao final deste encontro, o projeto do Vertical Slice deve conter:

- O `GameManager` e o `SaveManager` da Semana 4, sem nenhuma alteração.
- Uma Scene `scenes/interactables/Door.tscn`, com script `door.gd` implementando o contrato `Interactable (interact())`.
- Um `InteractionComponent` (`scripts/components/interaction_component.gd`) no Player, detectando objetos via `Area3D` e chamando `interact()` sem conhecer o tipo concreto do objeto detectado.
- Uma nova ação `interagir` no Input Map.

Segundo o `PROJECT_ARCHITECTURE.md` (seção 6, Módulo 2), este resultado corresponde ao item "Contrato Interactable (interface via duck typing)" do roadmap. O Encontro 2 desta semana adiciona o Signal disparado pela interação e conecta esse Signal a uma reação concreta (a porta abrindo).

Desafio

Cada estudante adapta a Scene `Door.tscn` para reagir de forma própria à chamada de `interact()`, sem alterar o `InteractionComponent` — por exemplo, trocar a mensagem impressa, alternar uma variável de estado (`var aberta: bool = false`) ou mudar a cor do

`MeshInstance3D` a cada interação. A reação deve continuar acontecendo inteiramente dentro da função `interact()` da `Door`.

Checklist

- ☐ Discussão sobre comunicação acoplada × contrato realizada
- ☐ Scene `Door.tscn` criada em `scenes/interactables/`, com `interact()` implementado
- ☐ `InteractionComponent` criado em `scripts/components/`, detectando via `Area3D`
- ☐ Ação `interagir` adicionada ao Input Map
- ☐ Chamada de `interact()` testada com sucesso a partir de pelo menos duas instâncias de `Door.tscn`
- ☐ Desafio (reação própria da `Door`) implementado

Glossário

- **Interactable (contrato):** comportamento esperado — responder a `interact()` — que qualquer Node pode implementar sem herdar de uma classe comum, verificado via `has_method("interact")`.
- **Duck typing:** checagem de capacidade de um objeto (se ele possui um método ou propriedade) em tempo de execução, sem depender de uma hierarquia de tipos declarada.
- **InteractionComponent:** Component do Player responsável por detectar objetos interativos próximos via `Area3D` e chamar o contrato `Interactable`, sem conhecer o tipo concreto do objeto.

Referências

- Godot Documentation — GDScript (métodos, `has_method`):
<https://docs.godotengine.org/en/stable/tutorials/scripting/gdscript/index.html>
- Godot Documentation — Area3D:
https://docs.godotengine.org/en/stable/classes/class_area3d.html
- Orchestrator — Documentação oficial: <https://orchestrator.cratercrash.space/>
- Unity Manual (consulta comparativa) — Interfaces em C#:
<https://docs.unity3d.com/Manual/interface.html>
- Canais recomendados para consulta complementar (não substituem a documentação oficial):
GDQuest (<https://www.youtube.com/@GDQuest>), Clear Code
(<https://www.youtube.com/@ClearCode>)